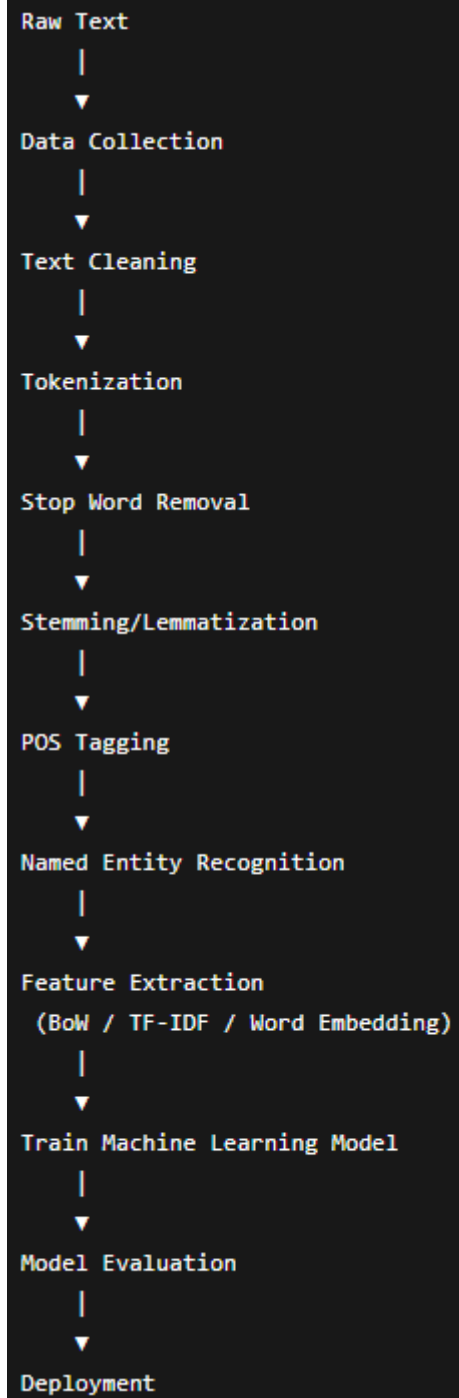




1. Text Collection:

- Gather raw text data from various sources like websites, social media, etc.

2. Exploratory Data Analysis (EDA):

Perform EDA to understand the structure and content of the dataset.

- Class Distribution:** To visualize the distribution of target classes (e.g., Sentiment: Positive, Negative, Non).
- Word Frequency:** Create a list of words and their frequencies to identify the most frequent words in the dataset.
- Word Cloud:** To visualize the words and their frequencies in a cloud format.
- Sentiment Distribution:** To visualize the distribution of sentiment labels in the dataset (if labels are available).

1. Data Collection



2. Data Understanding



3. Exploratory Data Analysis (EDA)



4. Text Preprocessing



5. Feature Engineering



6. Train-Test Split



7. Model Building



8. Model Evaluation



9. Model Improvement



10. Deployment

websites, social media, etc.

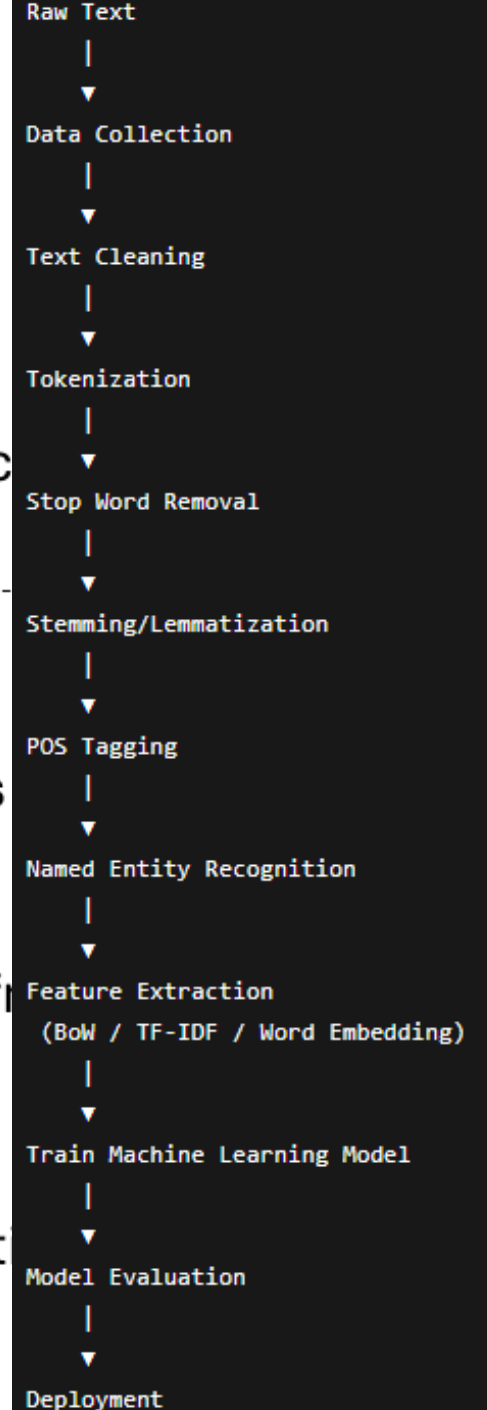
dataset.

target classes

the most frequent

dataset.

the sentiment



3.Text Preprocessing:

Clean and prepare the text data for model input.

- **Cleaning:** Messing Values, Duplicates.
- **Tokenization:** Split the text into words or sentences.
- **Lowercasing:** Convert all text to lowercase to standardize it.
- **Stop word Removal:** Remove common words like "and," "the," etc., that don't carry significant meaning.
- **Lemmatization/Stemming:** Reduce words to their base or root form (e.g., "running" to "run").
- **Noise Removal:** Remove punctuation, HTML tags, special characters, etc.

4. Feature Extraction:

Convert text data into numerical representations suitable for model input.

- **Bag of Words (BoW):** Represent text as a matrix of word counts.
- **TF-IDF:** Use Term Frequency-Inverse Document Frequency to represent text, which accounts for the importance of words in a document relative to the corpus.
- **Word Embeddings:** Use pre-trained embeddings like Word2Vec, GloVe, or contextual embeddings like BERT to capture semantic meaning.

5. Model Selection:

Choose the appropriate model based on the NLP task at hand (e.g., classification, sentiment analysis, language generation).

- For classification tasks, models like logistic regression, SVM, or neural networks can be used.
- For sequence-based tasks, recurrent neural networks (RNNs) or transformer-based models like BERT are popular choices

6. Model Training:

Train the selected model using the preprocessed and vectorized text data.

7. Model Evaluation:

Evaluate the model performance using metrics appropriate for the task.

- Classification tasks: Accuracy, Precision, Recall, F1-Score.
- Sentiment analysis: Confusion matrix, accuracy.

load Dataset

```
import pandas as pd
```

```
data =pd.read_csv("/content/drive/MyDrive/Colab Notebooks/  
Machine_Learning_Professional/NLP/Toxic_Dataset - Copy.csv")
```

Data Understanding
before analysis , understand

Dataset Shape



1 data.shape



(3014, 8)

Data Type

```
1 data.dtypes
```

```
0
```

```
Comment object
```

```
Toxic int64
```

```
dtype: object
```

first Records

```
1 data.head()
```

	Comment	Toxic
0	Madaxweyn guuleeyso inshaa alaah	0
1	Wuxuushnimadiyo qabyaaladaada warxumadaada war...	1
2	Masha Allaah Guul gacmahoo leys qabsado	0
3	Ka joog Xasanoow dhiiqaad lugaha kula jirtayee...	0
4	Mudane madaxweyne HSM dalki iyo dadki ba ku te...	0



Exploratory Data Analysis (EDA)

EDA helps understand the dataset before cleaning.

A. Check Missing Values

Why?

Missing text may cause errors.

```
data.isnull().sum()
```

B. Check Duplicate Records

Why?

Duplicates can bias the model.

```
data_duplicated=data.duplicated().sum()
```

```
print(data_duplicated)
```

Target Variable Distribution

Why?

Detect class imbalance.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
sns.countplot(data=data,x="Toxic")
plt.show()
```

```
ax = sns.countplot(data=data, x="Toxic", hue="Toxic",
palette="viridis", legend=False)
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.
get_width() / 2., p.get_height()),
ha='center', va='center', fontsize=11,
color='black', xytext=(0, 5),
textcoords='offset points')
plt.title('Distribution of Toxic/Non-Toxic Comments')
plt.xlabel('Toxic (0: Non-toxic, 1: Toxic)')
plt.ylabel('Count')
plt.show()
```


D. Text Length Analysis

Why?

Understand document size.

```
data['word_count'] = data['Comment'].apply(lambda x: len(str(x).split(" ")))
```

```
data[['Comment', 'word_count']].head()
```

```
data['word_count'].describe()
```

E. Distribution of Word Counts

```
plt.hist(data['word_count'])
```

```
plt.show()
```

F. Most Frequent Words

Why?

Identify dominant vocabulary.

```
from collections import Counter
```

```
all_words = " ".join(data['Comment'])
```

```
Counter(all_words.split()).most_common(20)
```

G. Word Cloud

Why?

Visualize common words.

```
from wordcloud import WordCloud
```

```
wordcloud = WordCloud(  
    width=800,  
    height=400  
).generate(all_words)
```

```
plt.imshow(wordcloud)  
plt.axis("off")
```

H. N-Gram Analysis

Unigram

Single word:

- Good
- Service
- product

Bigram

Two words:

- good service
- customer support
- high quality

Trigram

Three words:

- very good service
- easy to use

Most Frequent Unigrams

```
from sklearn.feature_extraction.text import CountVectorizer
# Unigram
vectorizer = CountVectorizer(ngram_range=(1,1))

X = vectorizer.fit_transform(data['Comment'])

word_counts = X.sum(axis=0).A1

unigram_freq = pd.DataFrame({
    'Word': vectorizer.get_feature_names_out(),
    'Frequency': word_counts
})

unigram_freq = unigram_freq.sort_values(
    by='Frequency',
    ascending=False
)

print(unigram_freq.head(10))
```

	Word	Frequency
11673	waa	707
5984	iyō	654
6753	ku	584
6236	ka	513
9045	oo	446
410	ah	415
5664	in	363
6981	la	358
10759	soo	262
3622	ee	262

Most Frequent Bigrams

```
vectorizer = CountVectorizer(  
    ngram_range=(2,2)  
)  
  
X = vectorizer.fit_transform(data['Comment'])  
  
bigram_counts = X.sum(axis=0).A1  
  
bigram_freq = pd.DataFrame({  
    'Bigram': vectorizer.get_feature_names_out(),  
    'Frequency': bigram_counts  
})  
  
bigram_freq = bigram_freq.sort_values(  
    by='Frequency',  
    ascending=False  
)  
  
print(bigram_freq.head(10))
```

	Bigram	Frequency
13740	insha allah	67
33	15 cod	48
12015	hassan sheikh	41
29380	waa in	39
10469	guul iyo	37
13319	in la	35
13410	in uu	31
25685	see more	28
21950	mudane madaxweyne	27
32498	xasan sheekh	27

Most Frequent Trigrams

```
vectorizer = CountVectorizer(  
    ngram_range=(3,3)  
)
```

```
X = vectorizer.fit_transform(data['Comment'])
```

```
trigram_counts = X.sum(axis=0).A1
```

```
trigram_freq = pd.DataFrame({  
    'Trigram': vectorizer.get_feature_names_out(),  
    'Frequency': trigram_counts  
})
```

```
trigram_freq = trigram_freq.sort_values(  
    by='Frequency',  
    ascending=False  
)
```

```
print(trigram_freq.head(10))
```

	Trigram	Frequency
12613	hassan sheikh mohamud	22
10889	guul iyo gobanimo	21
312	aad iyo aad	17
12432	hampalyo hampalyo hampalyo	17
800	abdirahman abdishakur warsame	16
29215	telefishinka qaranka soomaaliyeed	13
6843	dalka iyo dadka	13
21011	madaxweyne hassan sheikh	10
10891	guul iyo gobonimo	9
34037	xasan sheekh maxamuud	9

J. Vocabulary Size

Why?

Measure language richness.

```
1  vocabulary = set()
2
3  for text in data['Comment']:
4      | | vocabulary.update(text.split())
5
6  print(len(vocabulary))
```

15435

15,435 unique words

Common Cleaning Tasks

- Convert text to lowercase
- Remove punctuation
- Remove numbers
- Remove URLs
- Remove special characters
- Remove extra spaces

1. Convert Text to Lowercase

Good SERVICE Product

```
data['Comment'] = data['Comment'].str.lower()
```

good service product

2. Remove Punctuation good service!!!

```
import string
data['Comment'] = data['Comment'].apply(
    lambda x: ''.join(char for char in x if char not in string.punctuation)
)
```

3. Remove Numbers

```
import re

data['Comment'] = data['Comment'].apply(
    lambda x: re.sub(r'\d+', '', x)
)
```


4. Remove URLs

```
import re

data['Comment'] = data['Comment'].apply(
    lambda x: re.sub(r'http\S+|www\S+|https\S+', '', x)
)
```

5. Remove Special Characters

```
import re

data['Comment'] = data['Comment'].apply(
    lambda x: re.sub(r'^a-zA-Z\s', '', x)
)
```

6. Remove Extra Spaces

```
data['Comment'] = data['Comment'].str.strip()  
data['Comment'] = data['Comment'].str.replace(  
    r'\s+',  
    ' ',  
    regex=True  
)
```

Tokenization

What is Tokenization?

Splits text into individual words (tokens).

Tokenization (Somali)

my column is Comment

```
data['tokens'] = data['Comment'].str.split()
```

```
print(data['tokens'].head())
```

Stopword Removal (Somali)

```
somali_stopwords = [  
    'iyo', 'oo', 'ama', 'waa', 'ay', 'ka', 'ku', 'la',  
    'si', 'uu', 'u', 'ah', 'in', 'aan', 'wax'  
]  
  
data['tokens_no_stopwords'] = data['tokens'].apply(  
    lambda words: [word for word in words if word.lower() not in  
        somali_stopwords]  
)  
  
print(data['tokens_no_stopwords'].head())
```

Stemming

- **What is Stemming?**
- Reduces words to their root form.

Original Word	Stemmed Word
playing	play
played	play
plays	play
studies	studi
studying	studi

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
data['stemmed_tokens'] = data['tokens_no_stopwords'].apply(  
    | lambda words: [stemmer.stem(word) for word in words]  
    )
```

```
print(data['stemmed_tokens'].head())
```

Feature Extraction Methods in NLP

- Feature extraction converts text into numerical representations that machine learning models can understand.
- There are **two major categories**:
 1. Traditional Feature Extraction
 2. Embedding-Based Feature Extraction
 3. N-Gram Features : Instead of single words, use groups of words.
 4. Word Embeddings (Word2Vec, GloVe, FastText)
 5. Others

Bag of Words (BoW)

- Counts how many times each word appears in a document.
- **How it works:** Represents text as a vector of word counts or frequencies, ignoring grammar and word order.
- **When to use:**
 - Simple text classification tasks
 - When word order doesn't matter (spam detection, sentiment analysis)
 - Small to medium datasets with limited vocabulary
- **Sentence 1:** I love NLP
- **Sentence 2:** I love Machine Learning
- **Vocabulary:** [I, love, NLP, Machine, Learning]

Sentence	I	love	NLP	Machine	Learning
S1	1	1	1	0	0
S2	1	1	0	1	1

Raw text



"I love NLP"

"I love AI"

"I hate AI"



Vocabulary

[I, love, NLP, AI, hate]



BoW Matrix

	I	love	NLP	AI	hate
C1	1	1	1	0	0
C2	1	1	0	1	0
C3	1	0	0	1	1



Machine Learning Model

(Logistic Regression, SVM, Naive Bayes, etc.)

TF-IDF (Term Frequency–Inverse Document Frequency)

- Most Popular Traditional Method
- **How it works:** Weighs words by how frequently they appear in a document vs. how rare they are across all documents. Reduces the influence of very common words.
- **When to use:**
 - Document retrieval and search engines
 - When you need to highlight discriminative words
 - Information retrieval, keyword extraction
- **Bag of Words:** counts words only.
- **TF-IDF:** counts words and measures how important they are.

- **Example :** Suppose we have 3 documents:
- **Document 1 :** I love NLP
- **Document 2 :** I love AI
- **Document 3 :** I hate AI
- **Step 1: Vocabulary**
- Unique words:
 - I
 - Love
 - NLP
 - AI
 - hate:
- **Step 2: Calculate TF (Term Frequency)**
 - $TF = \text{Number of times word appears in document} \div \text{Total words in document}$

Document 1
I love NLP
Total words = 3

TF = Number of times word appears in document ÷ Total words in document

Word	Count	TF
I	1	1/3 = 0.333
love	1	1/3 = 0.333
NLP	1	1/3 = 0.333

Document 2
I love AI

Word	TF
I	0.333
love	0.333
AI	0.333

Document 3
I hate AI

Word	TF
I	0.333
hate	0.333
AI	0.333

- **Step 3: Calculate IDF (Inverse Document Frequency)**

- Formula: $IDF = \log(\text{Total Documents} / \text{Number of Documents Containing Word})$

- Total documents = 3

- **Word: I**

- Appears in: D1,D2, D3

- Document Frequency (DF) = 3

- $IDF = \log(3/3)$
= $\log(1)$
= 0

- **Word: love** Appears in: D1,D2

- DF = 2

- $IDF = \log(3/2)$
= 0.176

- **Word: NLP** : Appears only in: D1

- DF = 1

- $IDF = \log(3/1)$
= 0.477

- **Word: AI** Appears in: ,D2 ,D3

- DF = 2

- IDF = 0.176

- **Word: hate** Appears only in: D3

- DF = 1

- IDF = 0.477

Step 4: Calculate TF-IDF

Document 1 :I love NLP

$TF\text{-}IDF = TF \times IDF$

Word	TF	IDF	TF-IDF
I	0.333	0	0
love	0.333	0.176	0.059
NLP	0.333	0.477	0.159

Document 2 :I love AI

Word	TF	IDF	TF-IDF
I	0.333	0	0
love	0.333	0.176	0.059
AI	0.333	0.176	0.059

Document 3 :I hate AI

Word	TF	IDF	TF-IDF
I	0.333	0	0
hate	0.333	0.477	0.159
AI	0.333	0.176	0.059

Final TF-IDF Matrix

Docume nt	I	love	NLP	AI	hate
D1	0	0.059	0.159	0	0
D2	0	0.059	0	0.059	0
D3	0	0	0	0.059	0.159

Word2Vec

- Developed by Google.
- Concept
 - Learns word meaning from surrounding words.
 - Words with similar meanings have similar vectors.
- Example
 - King $\approx$ Queen
Doctor $\approx$ Physician
- Vector Example:
- King = [0.23, 0.87, 0.45, ...]
- Advantages
 - Captures semantic meaning
 - Dense vectors
 - Better than TF-IDF
- Disadvantages
 - One vector per word
 - Context independent

GloVe

- Developed by Stanford.
- **Concept**
 - Uses global word co-occurrence statistics.
- **Advantages**
 - Captures semantic relationships
 - Pre-trained models available

FastText

- Developed by Meta.
- **Concept**
 - Represents words using character n-grams.
- Example:
 - playing
played
player
- share subword information.
- **Advantages**
 - Handles rare words
 - Handles misspellings
 - Good for low-resource languages

Code of Bow

```
from sklearn.feature_extraction.text import  
CountVectorizer  
  
# Create BoW object  
bow = CountVectorizer()  
  
# Convert text into BoW features  
X_bow = bow.fit_transform(df['comment'])  
  
# Display as DataFrame  
bow_df = pd.DataFrame(  
    X_bow.toarray(),  
    columns=bow.get_feature_names_out()  
)  
  
print(bow_df)
```

TDF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF object
tfidf = TfidfVectorizer()

# Convert text into TF-IDF features
X_tfidf = tfidf.fit_transform(df['comment'])

# Convert to DataFrame
tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf.get_feature_names_out()
)

# Display first 5 rows
print(tfidf_df.head())

# Shape
print("Shape:", tfidf_df.shape)
```

Recommended TF-IDF Settings for Somali Comments

```
tfidf = TfidfVectorizer(  
    max_features=5000,  
    ngram_range=(1,2),  
    min_df=2,  
    max_df=0.95  
)  
  
X_tfidf = tfidf.fit_transform(df['comment'])
```

What these parameters mean

Parameter	Meaning
max_features=5000	Keep the 5000 most important words/features
ngram_range=(1,2)	Use both unigrams and bigrams
min_df=2	Ignore words appearing only once
max_df=0.95	Ignore words appearing in more than 95% of comments

Top Important Words in the Dataset

```
word_scores=tfidf_df.mean().sort_values(ascending=False)

top_words = pd.DataFrame({
    'Word': word_scores.index,
    'Average_TFIDF': word_scores.values
})

top_words.head(20)
```

Data Splitting

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Types of NLP Models

1. Traditional Machine Learning Model
2. Deep Learning Models
3. Transformer Models

- **Traditional Machine Learning Models**
 - These models cannot understand text directly.
 - They require feature extraction such as:
 - Bag of Words (BoW)
 - TF-IDF
 - N-Grams

- **Naive Bayes (NB)**
- **Type**
 - Supervised Learning
 - Classification
- **Used For**
 - Spam Detection
 - Sentiment Analysis
 - Toxic Comment Classification
- Document Classification
- **Advantages**
 - Very fast
 - Works well with TF-IDF
 - Good baseline model
- **Logistic Regression (LR)**
- **Type**
 - Supervised Learning
 - Classification
- **Advantages**
 - Easy to interpret
 - Strong performance on text data
 - Commonly used in research papers

- **Support Vector Machine (SVM)**

- **Type**

- Supervised Learning
- Classification

- **Advantages**

- High accuracy
- Excellent for sparse TF-IDF features

- **Disadvantages**

- Slower on very large datasets

- **Decision Tree**

- **Type**

- Supervised Learning
- Classification / Regression

- **Used For**

- Classification Tasks
- Rule-Based Predictions

- **Advantages**

- Easy to visualize

- **Disadvantages**

- Overfitting risk

- **Random Forest**
- **Type**
 - Supervised Learning
 - Classification / Regression
- **Advantages**
 - Better than a single Decision Tree
 - Reduces overfitting
- **XGBoost**
- **Type**
 - Supervised Learning
 - Classification / Regression
- **Used For**
 - Competition Projects
 - Advanced Classification
- **Advantages**
 - Very powerful
 - High accuracy

Deep Learning Models

- These models learn patterns automatically from text.
- Usually use:
 - Word2Vec
 - FastText
 - Embeddings
- **Artificial Neural Network (ANN)**
- **Type**
 - Deep Learning
 - Classification / Regression
- **Used For**
 - Basic NLP Tasks
 - Text Classification
- **Recurrent Neural Network (RNN)**
- **Type**
 - Deep Learning
 - Sequence Model
- **Used For**
 - Text Generation
 - Language Modeling
 - Sequence Prediction
- **Advantage**
 - Processes text sequentially

- **Long Short-Term Memory (LSTM)**
- **Type**
 - Deep Learning
 - Sequence Model
- **Used For**
 - Sentiment Analysis
 - Toxic Comment Detection
 - Machine Translation
- **Advantage**
 - Remembers long-term dependencies
- **Gated Recurrent Unit (GRU)**
- **Type**
 - Deep Learning
 - Sequence Model
- **Used For**
 - Similar tasks as LSTM
- **Advantage**
 - Faster than LSTM
 - Requires fewer parameters

- **CNN for NLP**
- **Type**
 - Deep Learning
 - Classification
- **Used For**
 - Text Classification
 - Sentiment Analysis
- **Advantage**
 - Fast training
 - Captures local patterns

- **Transformer Models**
 - Modern NLP models based on the Transformer architecture.
 - These are currently the most powerful NLP models.
- **BERT**
- (Bidirectional Encoder Representations from Transformers)
- **Type**
 - Transformer
 - Pretrained Language Model
- **Used For**
 - Text Classification
 - Question Answering
 - Sentiment Analysis
- **Advantage**
 - Understands context

```
1  # step7_predict_new.py
2
3  new_texts = [
4      "I want to build a chatbot using NLP",
5      "Neural networks can solve vision problems",
6  ]
7
8  new_cleaned = [preprocess(t) for t in new_texts]
9  new_tfidf = vectorizer.transform(new_cleaned)
10 new_preds = model.predict(new_tfidf)
11
12 for text, pred in zip(new_texts, new_preds):
13     label = "NLP-related" if pred == 1 else "General AI"
14     print(f" '{text}' → {label}")
```
